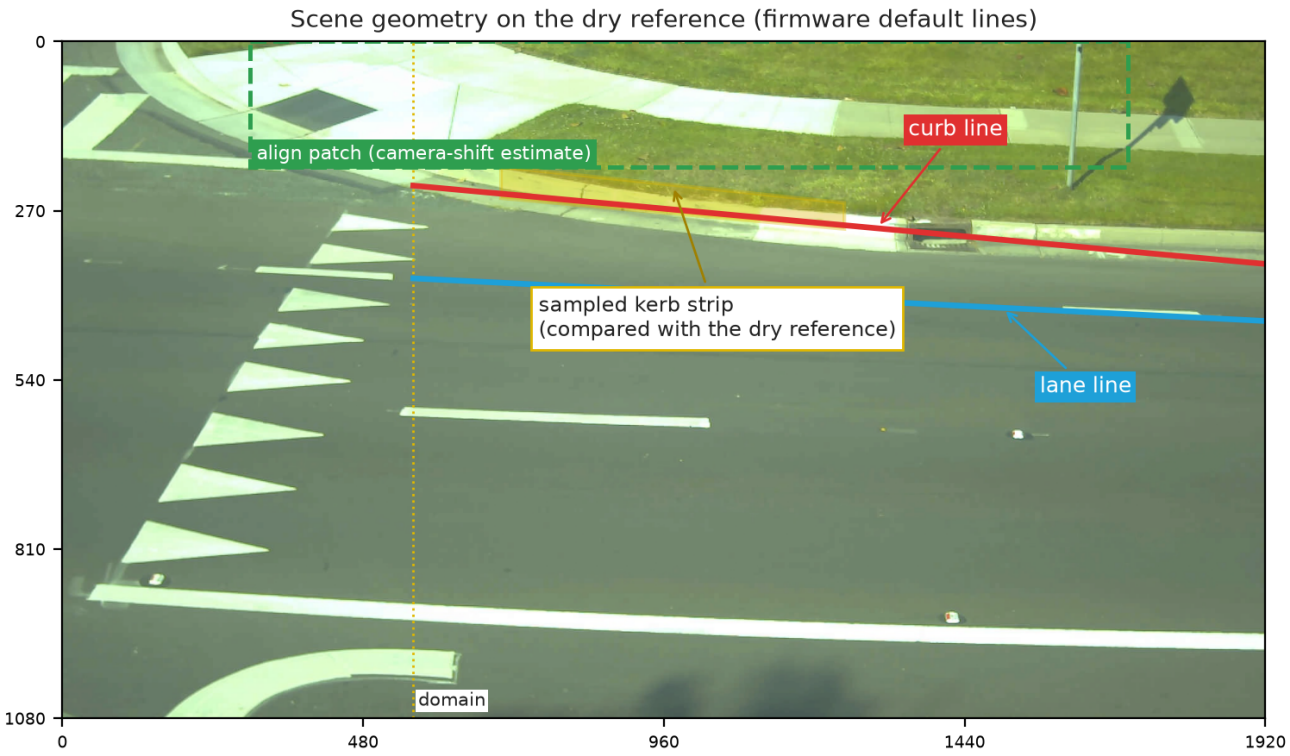
MS500-P4 Flood Simulator

Handbook

Run the camera firmware's flood-detection C code, unmodified, on recorded video and images: evaluate, regression-test and tune it on your PC.



Repository: D:\git\ms500-p4-simulator · generated 2026-09-30 from docs/HANDBOOK.md

The simulator runs the **MS500-P4 camera firmware's flood-detection C code**, unmodified, on video and image files instead of the live camera. It gives the same flood / no-flood decisions the camera would make, on your PC, from recorded footage.

Use it to:

- **evaluate** the detector on recorded footage and score it against ground-truth labels;
- **regression-test** firmware changes before flashing: did an edit change any decision?
- **debug and tune** visually: step through frames, move the curb/lane lines, see why a frame was judged the way it was.

It does **not** emulate the rest of the device: no web UI, MQTT, real uploads, or IMX500 sensor.

Contents

1. Setup
2. How the detector works (the 2-minute version)
3. Inputs
4. The GUI
5. The command line (`floodsim`)
6. `flood_sim.exe`: the firmware C code without Python
7. Files the simulator reads and writes
8. Recipes
9. When the firmware changes
10. Troubleshooting
11. Limits: where the simulator differs from the camera

1. Setup

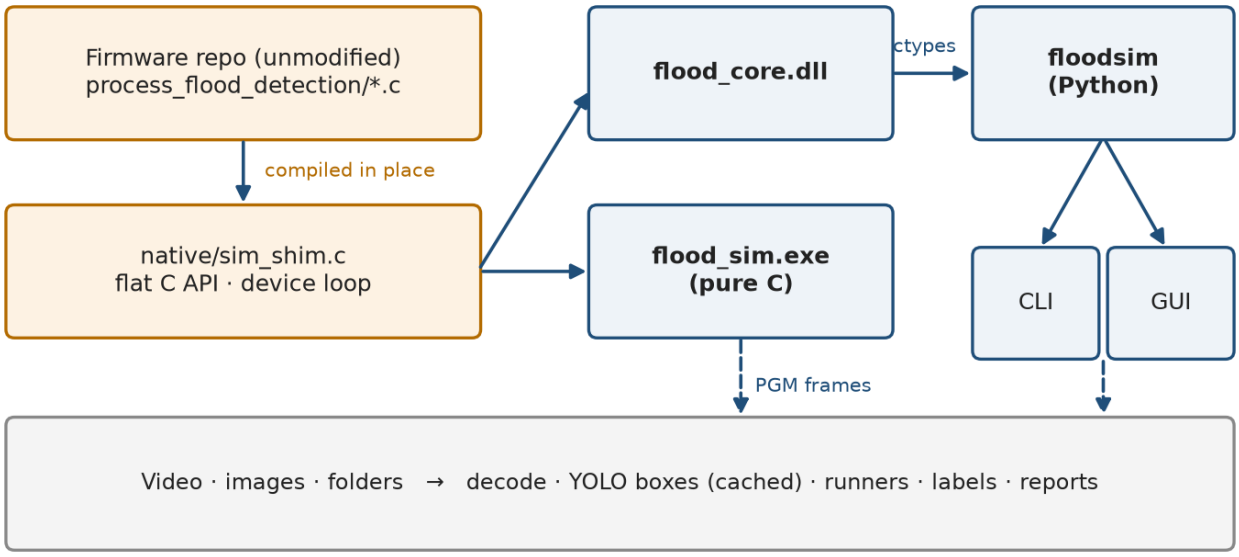


Figure 1 - How the pieces fit: the firmware C code is compiled in place into a DLL (used by Python) and a standalone exe.

Requirements (Windows)

Tool	Used for
MinGW-w64 `gcc`, `cmake` ≥ 3.20, `ninja` on PATH	compiling the firmware C code
Python 3.11 (`py -3.11`)	the simulator itself
The firmware repo at `D:\git\NETWORK_P4_FLOOD_DETECTION`	the C code being simulated

Install

In Git Bash:

```
cd /d/git/ms500-p4-simulator
py -3.11 -m venv .venv
.venv/Scripts/python -m pip install -e ".[dev,gui]"      # core + GUI + tests
.venv/Scripts/python -m pip install -e ".[onnx]"         # obstruction detector runtime (onnxruntime)
```

- **Put the device's traffic model at `models/traffic.onnx`.** It is the obstruction detector (not in git). Another model: pass `--model`; a `.pt` file runs through ultralytics (`[yolo]` extra).
- **The first run compiles the firmware code automatically.** Output goes to `build/native/` (`flood_core.dll`, `flood_sim.exe`).
- **Firmware checked out somewhere else?** Point the simulator at it:

```
export FLOOD_FIRMWARE_DIR=/d/path/to/NETWORK_P4_FLOOD_DETECTION
```

Check the install

```
.venv/Scripts/python -m pytest -q                 # all Python tests should pass
ctest --test-dir build/native --output-on-failure  # all C tests should pass
```

2. How the detector works (the 2-minute version)

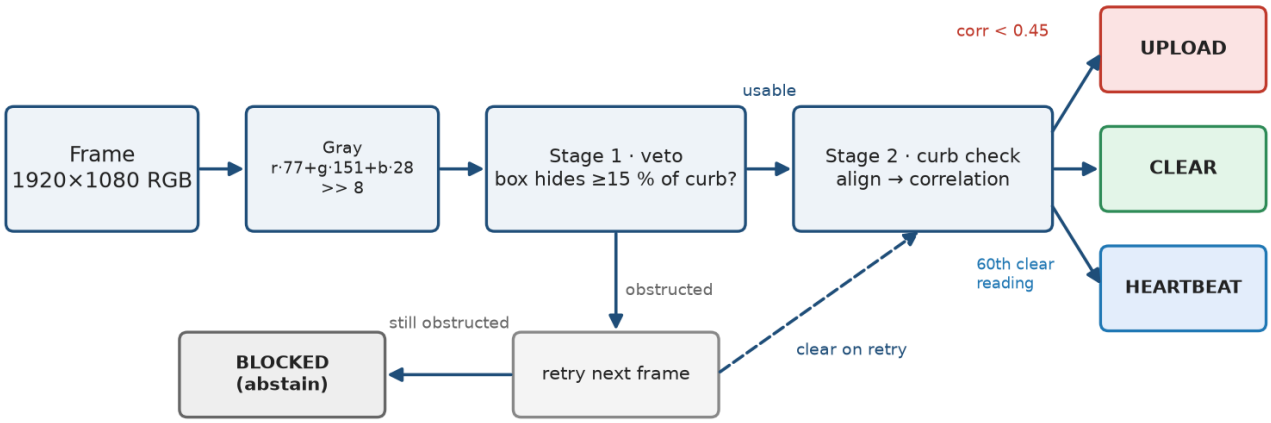


Figure 2 - One device sample: veto first, then the curb check; an obstructed frame is retried once, then reported as BLOCKED.

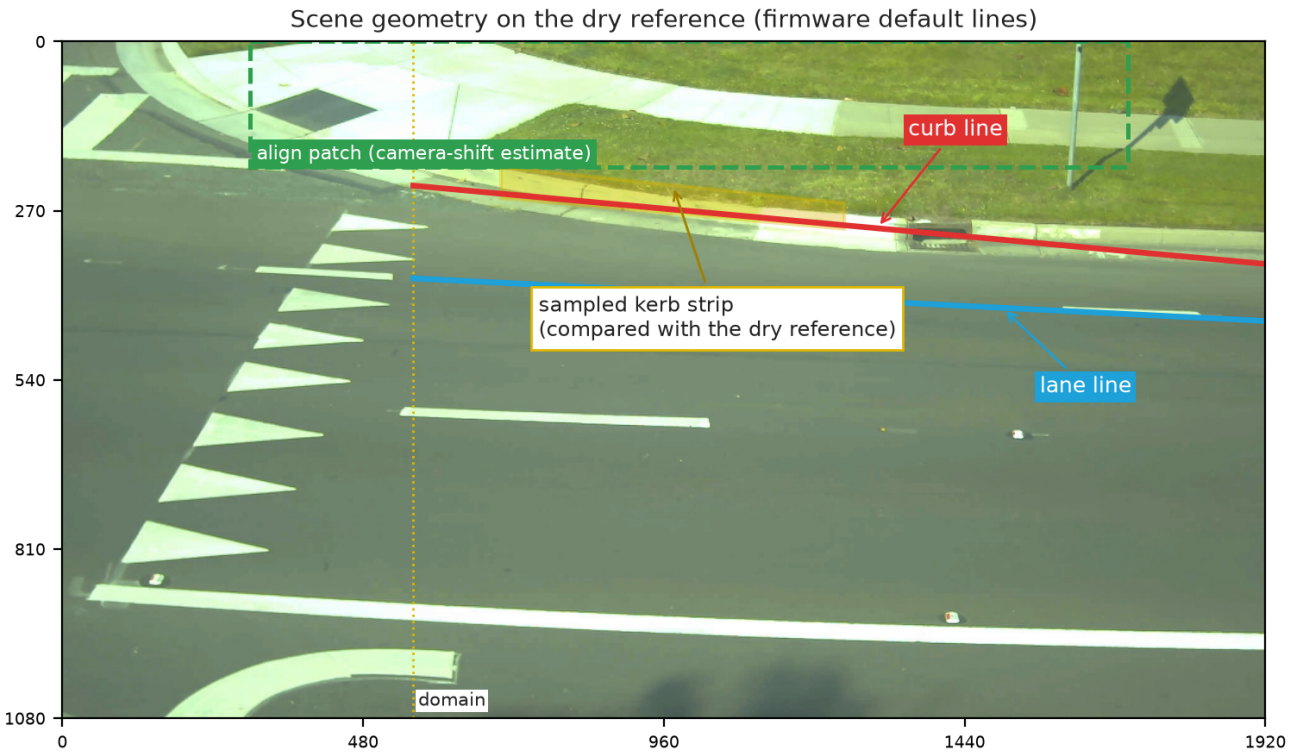


Figure 3 - What the firmware looks at: curb and lane lines, the sampled kerb strip and the alignment patch.

You need these concepts to read the simulator's output.

The camera asks one question: "Is the curb still there?" Rain darkens a street but leaves the curb's edge and shadow intact. Standing water covers them. So the detector compares a thin **strip along the curb** against a **dry reference** image, using normalized correlation, which ignores changes in brightness and contrast.

Term	Meaning
Dry reference	A frame of the same scene when it was dry. On the device it is captured by "Capture Baseline".
Reference gallery	Up to 8 dry references, so a frame is compared against whichever one matches the current lighting (day, dusk, night…). Every frame is scored against all of them and the best match counts. **Manual** references are the ones you add; the device adds **learned** (auto) ones itself when a sample is confidently dry (0.65 ≤ correlation < 0.95) but not yet well covered, **and** at least one manual reference still scores it ≥ 0.55 (so learning cannot follow a slowly rising flood). After 5 samples in a row between 0.45 and 0.65, it logs a *no confident match* notice: time to capture a baseline for that lighting.
Geometry	Two lines you place on the image: the **curb line** (the curb's foot) and the **lane line** (below it, on the road). The sampled **kerb strip** is derived from them.
Correlation	−1 … 1. It measures how much the curb strip still looks like the dry reference. **Below 0.45 = submerged.**
Severity	From correlation: `none` ≥ 0.45 > `minor` ≥ 0.05 > `moderate` ≥ −0.10 > `severe`.
Shift (dx, dy)	Alignment against a patch of pavement above the curb, which corrects for the camera mount sagging over time.
Veto (Stage 1)	People and vehicles found by the object detector. If a box hides ≥ 15 % of the curb, the frame is **not usable**: something is in the way, and that is not evidence of a flood.
Retry	If the first frame is obstructed, the device takes a second frame and tries again.

What one device sample can report

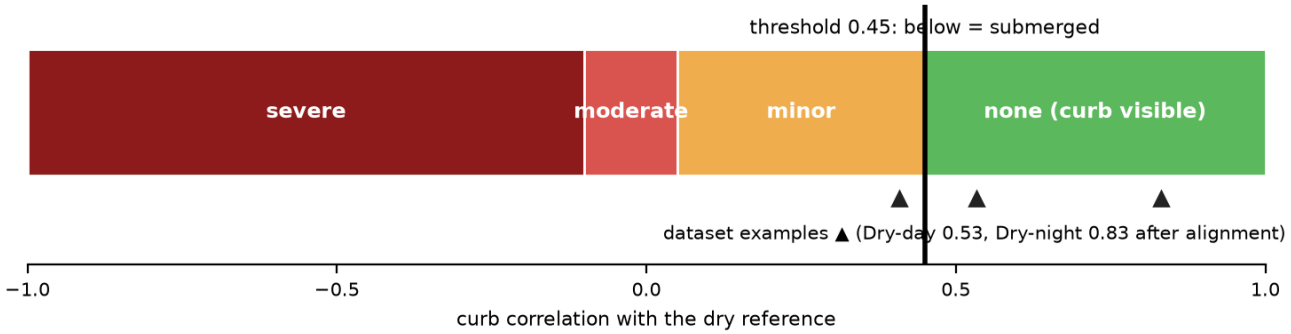


Figure 4 - Correlation, severity bands and the 0.45 threshold.

Outcome	Meaning	Device sends an image?
`upload`	Curb submerged: candidate flood	yes
`clear`	Curb visible: not flooded	no
`blocked`	Obstructed on every try. This is an **abstention**, never "no flood".	no
`heartbeat`	Every 60th reading without a flood: proof of life	yes
`skipped_no_ref`	No dry reference yet, so nothing was assessed	no

Three ways to run it

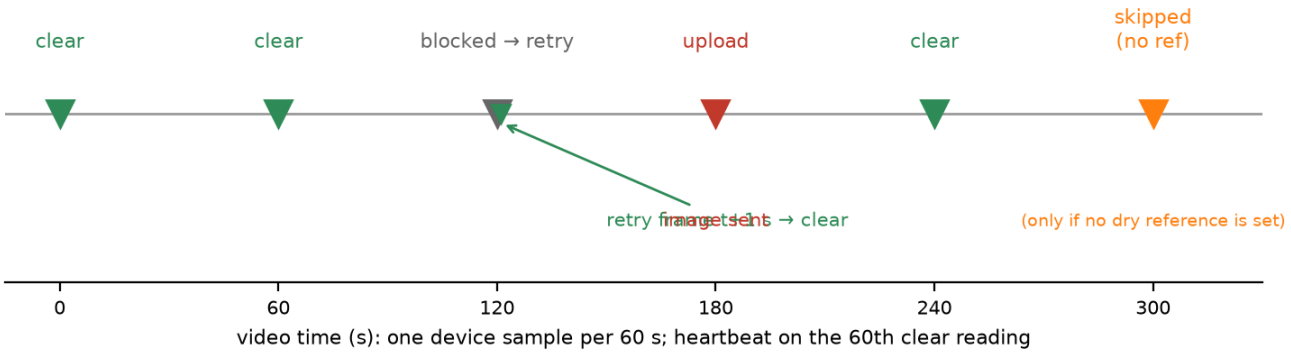


Figure 5 - Device mode on a video timeline: sample every 60 s, retry after 1 s when obstructed.

Mode	What it does	Use it for
device	One sample every 60 s of video time, with retry and heartbeat, exactly as the camera does	"What would the camera have reported on this footage?"
dense	Scores every frame (or every Nth) independently, with no retry or heartbeat	Correlation curves, threshold tuning, finding where things go wrong
manual (GUI only)	You pick the frame and press Sample or Score	Investigating single moments

3. Inputs

The simulator accepts a **video**, a **single image**, or a **folder of images**.

- **Resolution:** 1920×1080. Other 16:9 sizes are resized with a warning. Other aspect ratios are refused, because the curb geometry would not line up; `--allow-stretch` forces them through.
- **Video:** any format OpenCV reads (`.avi`, `.mp4`, …). Frame times come from the video's frame rate.
- **Single image:** one frame at t = 0.
- **Image folder:** frames are ordered by file name. Timing comes from a `_t<milliseconds>` part in the name, e.g. `f000012_t0000060000.png` = 60 s. Without that part, give a frame rate: `--fps` on the command line, or the GUI will ask for one.
- **"The frame at time t"** always means the latest frame whose timestamp is ≤ t. This is the frame a camera polled at t would have handed over.

4. The GUI

Start

```
.venv/Scripts/floodsim gui                    # empty window; open something from the File menu
.venv/Scripts/floodsim gui clip.avi           # open a video straight away
.venv/Scripts/floodsim gui frames/ --fps 4    # a folder without _t<ms> names
.venv/Scripts/floodsim gui clip.avi --no-detector   # don't load the detector (starts faster)
```

Layout

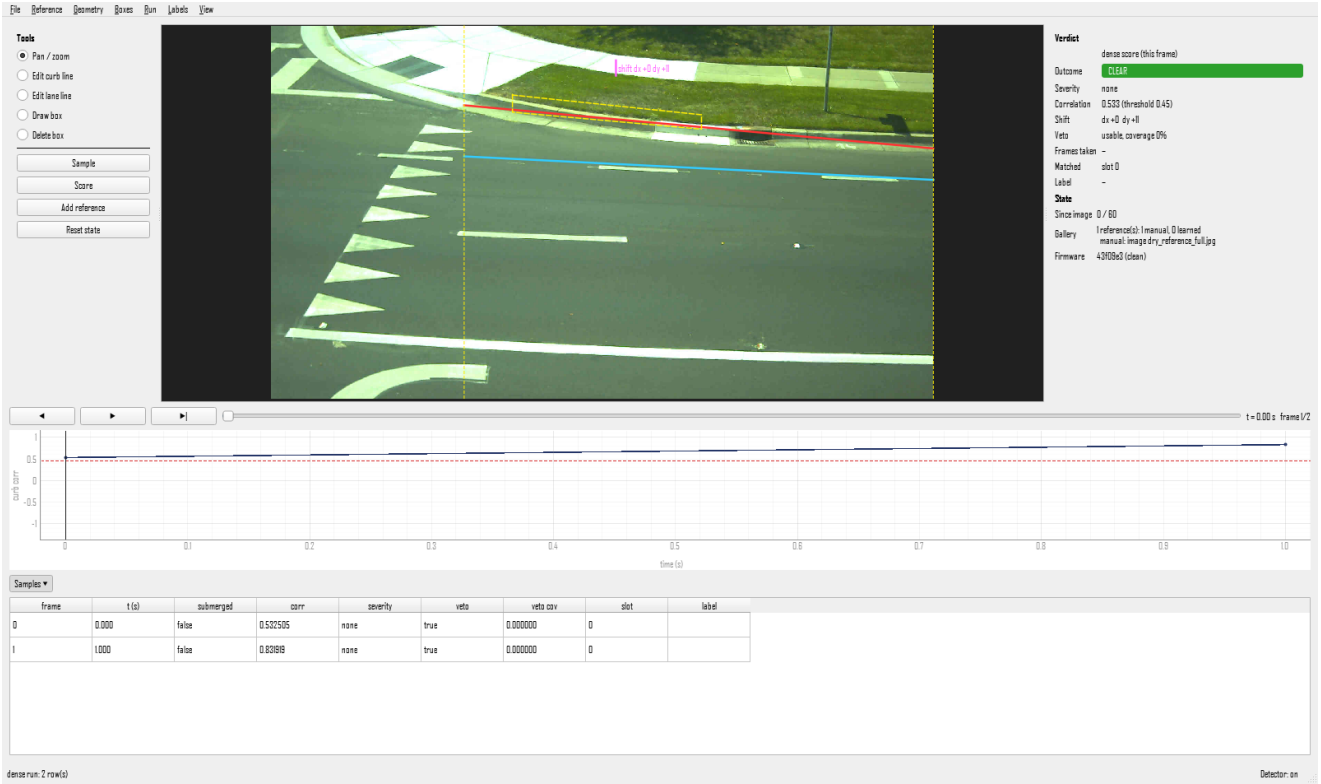


Figure 6 - The GUI (layout A) on the dataset frames with the dry reference and a dense run.

Overlays on the frame. Toggle each under **View**.

Overlay	Colour	Shows
Curb line	red	Where the curb foot is
Lane line	light blue	The reference line on the road below
Strip + domain	yellow, dashed	The exact strip of pixels the firmware samples, and the horizontal range where both lines are defined
Align patch (off by default)	green, dashed	The pavement patch used to estimate camera shift

Overlay	Colour	Shows
Boxes	colour per class; **dashed = edited by you**	Detections, labelled with class and % of the curb they hide
Shift vector	pink	The alignment shift, drawn 4× longer so it is visible

View ▸ Difference from reference shows how each pixel differs from the dry reference: bright means changed.

The plot shows correlation over time:

- **dots:** scored frames
- **red dashed line:** the 0.45 threshold
- **grey bars:** frames where the veto blocked the reading
- **triangles:** device samples (red = upload, green = clear, grey = blocked, blue = heartbeat)
- **coloured strip at the bottom:** labels, when loaded (brown = DRY, blue = WET, red = FLOODED, grey = INDETERMINATE)

Navigation: drag the slider, click the plot, use the ◀ / ▶| buttons, or press ← → on the keyboard. **Space** plays and pauses. Zoom the frame with the mouse wheel; double-click to fit it to the window.

First steps

1. **Open** footage: **File ▸ Open video / image / folder**.
2. **Add a dry reference.** Either:
   - go to a frame where the street is dry, then **Reference ▸ Add current frame** (or the *Add reference* button); or
   - **Reference ▸ Add image…**, e.g. `dataset/dry_reference_full.jpg`.

Every frame you visit is now scored, and the verdict panel shows the result. Adding another reference (say, a dusk frame) **adds** it to the gallery rather than replacing the first. The *Gallery* line lists what is in it; **Reference ▸ Clear all references** empties it. **View ▸ Show reference**, then a reference's entry, shows that manual reference in the main view, with the curb/lane overlays, in the grayscale the firmware compares against. Moving to any frame (or **Off**) returns to the footage. Learned references are not listed: they keep no image.

3. **Check the lines.** If the red and blue lines don't sit on the curb and road in your footage, fix them first (next section). Everything depends on them.

A day of recordings (folder of clips)

File ▸ Open folder… on a folder of videos named `record_YYYYMMDD_HHMMSS.avi` opens them as one timeline, each clip at its real start time (dates carry across midnight).

- The time axis shows the time of day; stretches with no footage are shaded. The readout shows the time, the clip and the frame.
- A **Device run** checks every *period* seconds of real time. Checks that fall between clips are skipped: no row, and the count appears as "checks skipped (no footage)" in the status bar and `summary.txt`. The *sample* numbers keep their place, so a jump in them marks a gap.
- The learned gallery and the heartbeat counter carry across clips, as on one device.
- Each clip keeps its own `.boxes.csv` / `.boxes.edited.csv`, shared with opening that clip alone.
- One edge case: an obstructed check in a clip's last second has its retry fall in the gap, so it comes out `blocked` (on the device the retry would have had a frame).
- Clip names must be `record_YYYYMMDD_HHMMSS`; a folder mixing videos and images is refused; if two clips overlap, the earlier one's overlapping frames are dropped with a warning.
- `floodsim export` (frames for `flood_sim.exe`) does not support folders of clips.

Manual mode

Button	What it does
Sample	A full device sample at the current frame. If it is obstructed, it retries on the frame 1 s later. It advances the heartbeat counter, adds a row to *Samples*, and **can learn a reference**, as the device does. *Matched* in the verdict panel shows which slot matched and whether a reference was learned or a *no confident match* notice fired.
Score	Stage 1 (veto) + Stage 2 (curb check) on this frame only, against the whole gallery. The device state is untouched and nothing is learned.
Add reference	Adds this frame to the gallery as a manual reference (the device's Capture Baseline).
Reset state	Sets the heartbeat counter ("since image") back to 0 and **forgets learned references**; manual ones stay.

Tuning the curb and lane lines

1. Choose **Edit curb line** (or **Edit lane line**). Handles appear on the line's two ends.
2. Drag a handle. The line follows the mouse; the yellow strip redraws when you let go.
3. The firmware's own rules are enforced. If you put the lane above the curb, or make the lines overlap too little horizontally, you get the firmware's error message and the line snaps back.
4. After an edit, the reference is **stale**: it was built under the old lines. A yellow banner says so. As on the device, **the old reference keeps being used** until you click **Rebuild reference**. Rebuilding rebuilds the manual references and drops learned ones: no frame is kept for them, here or on the device.
5. **Geometry ▸ Save…** writes the lines as JSON, in the same shape as the camera's web API. You can copy the values to the device, or reuse them with `floodsim run --geometry`. **Geometry ▸ Reset to firmware default** undoes all edits.

Obstruction boxes

- **With the detector available:** boxes are detected on each frame you visit and cached next to the input.
- **Draw box:** drag a rectangle, then pick a class (person, car, truck, bus, bicycle, motorcycle).
- **Delete box:** click inside a box.
- **Where edits go:** `<input>.boxes.edited.csv`. They override detected boxes for that frame and show dashed.
- **Boxes ▸ Revert frame to detector boxes** removes your edits for the current frame.
- **Without it** (status bar: *Detector: off*): only cached and edited boxes exist. Frames with neither have no boxes, so the veto can never trigger on them.

Batch runs

- **Run ▸ Device run…** asks for the sample period (default: the firmware's 60 s) and the retry gap (default 1 s). **On a short clip, shorten the period** (e.g. 5 s): at 60 s a 5-minute clip gives only 5 samples, too few to see references being learned or the 5-sample notice.
- **Run ▸ Dense run…** asks for "score every Nth frame". Dense runs never learn references.

A device run starts from your manual references only, so the same inputs always give the same rows (`admitted`, `gallery_count`, `slot`). **When it finishes, the session keeps the gallery the run ended with**, as the device keeps what it learns: the *Gallery* line then counts the learned references, and the curve, verdict, **Score** and **Dense run** score against them. Run **Dense run** again after a device run to redraw the curve with what was learned. **Reset state** goes back to the manual references only.

A dense run scores against the session's gallery as it is, learned references included, and never changes it. `run.json` records how many learned references it used.

A progress bar and **Cancel** appear. When the run finishes, results appear in the plot and the *Samples* table. **Double-click a row** to jump to its frame. The status bar reports:

- **reference rebuilt:** the reference was stale, so it was rebuilt first. Batch runs always use a reference matching the current lines, as the command line does.
- **N frame(s) had no boxes:** no detector and no cache for those frames. Their veto is missing.
- **(cancelled):** partial results.

While a run is going, other actions queue and run after it.

Labels and export

- **Labels ▸ Load labels…** loads a ground-truth CSV (format in §7). It appears as the coloured strip and in the table's *label* column.
- **File ▸ Export run…** writes the last batch run to a new folder. The folder has the same layout as `floodsim run` output: `run.json`, `samples.csv`, `summary.txt`, and `uploads/` for device runs. With labels loaded, `summary.txt` includes the scoring.

A GUI run and a command-line run with the same inputs and settings produce identical `samples.csv`.

5. The command line (`floodsim`)

Use the command line for anything repeatable: evaluations, regression checks, scripts.

`floodsim run`: a device or dense run

```
floodsim run INPUT [--mode device|dense] [--reference IMG ...] [--reference-at SECONDS ...]
                   [--geometry GEO.json] [--labels LABELS.csv] [--out DIR]
                   [--period 60] [--retry-gap 1] [--every 1]
                   [--no-detector] [--model models/traffic.onnx] [--conf 0.5]
                   [--fps F] [--allow-stretch] [--no-rebuild]
```

Option	Meaning
`--mode`	`device` (default) or `dense`
`--reference` / `--reference-at`	Manual dry references: image files, or the input's own frames at those times. Both can be repeated and combined (up to 8 in total), e.g. one per lighting condition. Dense mode requires at least one. Device mode without one reports every sample as `skipped_no_ref`, as the device would.
`--geometry`	Curb/lane lines (JSON, §7). The default is the firmware's built-in lines.
`--labels`	Score the results against ground truth (§7)
`--out`	Output folder. The default is `out/<input>-<mode>-<timestamp>`. It must not already contain files.
`--period`, `--retry-gap`	Device mode timing. `--period` defaults to the firmware's 60 s; shorten it to get more samples (and more reference learning) out of a short clip.
`--every`	Dense mode: score every Nth frame
`--no-detector`	No obstruction boxes at all, including edited ones
`--model`, `--conf`	Detection model (`.onnx`: the device's traffic model; `.pt`: an ultralytics YOLO) and score threshold. The defaults match the device: `models/traffic.onnx`, 0.5, NMS 0.20 across classes, motorcycles dropped.
`--no-rebuild`	Use the existing DLL even if the firmware changed. `run.json` then records `"stale": true`.

Examples:

```
# What would the camera have reported on this clip?
floodsim run clip.avi --reference dry.jpg

# Correlation for every frame, scored against labels
floodsim run clip.avi --mode dense --reference dry.jpg --labels clip.labels.csv

# Custom geometry, every 4th frame
floodsim run clip.avi --mode dense --reference dry.jpg --geometry site42.json --every 4

# Day and night baselines; sample every 5 s to see learning on a short clip
floodsim run dusk.avi --reference noon.jpg --reference night.jpg --period 5
```

`floodsim detect`: fill the box cache in advance

```
floodsim detect clip.avi [--every 1] [--model models/traffic.onnx] [--conf 0.5]
```

This runs the detector once and saves the boxes to `clip.avi.boxes.csv`. Later runs reuse the cache, including on machines without the detector. The cache belongs to the model, confidence, NMS, and input file (name, size,

modification time); if any of those change, it is set aside as `….boxes.csv.stale` and detection starts again.

`floodsim compare`: did anything change?

```
floodsim compare out/before out/after
```

This lists every sample where the outcome, severity, chosen frame, veto, submerged flag, matched gallery slot, or learned/notice flag differs, or where the correlation differs by more than 1e-6. It exits with code 1 if anything differs and 0 if the runs are identical, so you can use it in scripts. Each argument can be a run folder, a `samples.csv`, or `flood_sim.exe` output (`.jsonl`).

`floodsim export`: frames for `flood_sim.exe`

```
floodsim export clip.avi --reference dry.jpg --out out/exp [--every 1] [--no-detector]
```

This writes `frames/f<idx>_t<ms>.pgm` (grayscale, converted exactly as the camera does), `boxes.csv`, and `reference.pgm`. Repeat `--reference` for more: they become `reference_1.pgm`, `reference_2.pgm`, and so on.

`floodsim gui`

See §4.

6. `flood_sim.exe`: the firmware C code without Python

`build/native/flood_sim.exe` is the same firmware code as a standalone C program. It is useful to rule out the Python layer, or to check a result without a Python environment.

```
floodsim export clip.avi --reference dry.jpg --out out/exp
build/native/flood_sim.exe --reference out/exp/reference.pgm --frames out/exp/frames \
    --boxes out/exp/boxes.csv --mode dense > out/exe.jsonl
floodsim compare out/some-python-run out/exe.jsonl        # expect 0 differences
```

Options: `--reference R.pgm [--reference R2.pgm ...] --frames DIR|LIST [--boxes B.csv] [--geometry G.json] [--mode device|dense] [--period 60] [--retry-gap 1] [--every 1] [--fps 1]`. It prints one JSON line per sample, with the same fields as `samples.csv`. It reads only 1920×1080 PGM frames and does no video decoding and no detection. Keep its paths ASCII.

7. Files the simulator reads and writes

Run folder

File	Contents
`run.json`	Everything needed to reproduce the run: input (path, size, modification time), mode, parameters, geometry, reference source, box source, firmware commit and whether its flood code had uncommitted edits, and DLL version. GUI exports also record `cancelled` and `frames_without_boxes`.
`samples.csv`	One row per sample (device mode) or per scored frame (dense mode); columns below
`summary.txt`	Outcome counts, failed frame reads, references learned, *no confident match* notices, gallery size at the end (device mode), and scoring when labels were given
`uploads/`	Device mode only: the images the camera would have sent, named `<sample>_<label>.jpg`. Half resolution, with a 6-hex ID burned in, as on the device.

`samples.csv`, device mode: `sample, t_sec, frame_idx, outcome, frames_taken, has_reading, curb_correlation, severity, shift_dx, shift_dy, veto_usable, veto_coverage, veto_worst_class, since_image, gallery_slot, gallery_source, admitted, notice, gallery_count` (+ `label`).

- `t_sec` is the sample time.
- `frame_idx` is the frame actually used; −1 means no frame was available.
- `curb_correlation` is only meaningful when `has_reading` is true.
- `gallery_slot` / `gallery_source` are the reference that matched (`manual` or `auto`); −1 / `none` without a reading.

- `admitted` is true when this sample learned a new reference; `notice` when it raised the *no confident match* notice; `gallery_count` is the gallery size after the sample.

`samples.csv`, dense mode: `frame_idx, t_sec, veto_usable, veto_coverage, veto_worst_class, curb_correlation, submerged, severity, shift_dx, shift_dy, gallery_slot` (+ `label`).

Scoring (`summary.txt`)

- **Positive** = labelled `FLOODED`. **Negative** = `DRY` or `WET`. `INDETERMINATE` is left out.
- **Predicted flood** = outcome `upload` (device mode) or `submerged` (dense mode).
- **Blocked or vetoed samples are abstentions.** They are counted separately and never counted as negatives.
- The summary also reports:
  - the **margin**: the highest correlation among flooded samples and the lowest among dry/wet ones;
  - with zero false alarms, the **rule-of-three bound**: the false-alarm rate is only known to be below 3/N, not zero.

Labels CSV

Either time intervals (for videos):

```
start_sec,end_sec,label
0,120,DRY
120,300,WET
300,420,FLOODED
```

or file names (for image folders):

```
file,label
f000001_t0000060000.png,FLOODED
```

- Labels are `DRY`, `WET`, `FLOODED` or `INDETERMINATE`; case doesn't matter.
- Excel files saved as "CSV UTF-8", including empty `,,` rows, work.
- Gaps and overlaps between intervals produce a warning. Samples in a gap are unlabelled.

To convert the flooding-detection dataset's burst ground truth into this format:

```
.venv/Scripts/python tools/truth_to_labels.py \
    --truth   ../MS500_Flooding_Detection/dataset/truth.csv \
    --bursts  ../MS500_Flooding_Detection/dataset/bursts.json \
    --videos  ../MS500_Flooding_Detection/Videos --out labels/
```

Geometry JSON

This is the same shape as the camera's web API. Extra keys (e.g. `code`, `geometry_version`) are ignored:

```
{"curb": {"x0": 560, "y0": 230, "x1": 1920, "y1": 355},
 "lane": {"x0": 560, "y0": 378, "x1": 1920, "y1": 446}}
```

Box files (next to the input)

File	Written by	Notes
`<input>.boxes.csv`	Detector (`detect`, runs, GUI)	Cache. Starts with a `# meta:` line recording the model, confidence and input. Safe to delete.
`<input>.boxes.edited.csv`	GUI box tools	Your edits. They override the cache for those frames.
`<input>.boxes.csv.stale`	—	An older cache made with different settings, kept rather than deleted

8. Recipes

Evaluate the detector on a labelled clip

```
floodsim detect clip.avi                                    # once, if the detector is available
floodsim run clip.avi --mode dense  --reference dry.jpg --labels clip.labels.csv --out out/clip-dense
floodsim run clip.avi --mode device --reference dry.jpg --labels clip.labels.csv --out out/clip-device
```

Read `summary.txt` in each folder. Dense tells you how well the curb check separates flooded from not flooded. Device tells you what the camera would actually have reported and uploaded.

Check a firmware change before flashing

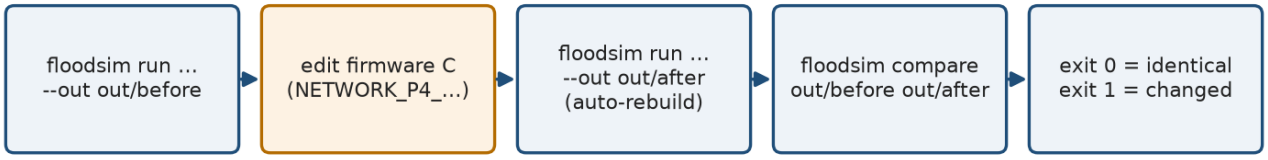


Figure 7 - Regression check before flashing.

```
floodsim run clip.avi --mode dense --reference dry.jpg --out out/before   # before the edit
# ... edit the firmware C code in NETWORK_P4_FLOOD_DETECTION ...
floodsim run clip.avi --mode dense --reference dry.jpg --out out/after    # rebuilds automatically
floodsim compare out/before out/after                                      # exit code 1 = something changed
```

Both `run.json` files record the firmware commit, so you can always tell which code produced which numbers.

Set up the lines for a new camera site

1. `floodsim gui site.avi`, go to a dry frame, click **Add reference**.
2. Use **Edit curb line** / **Edit lane line** until the yellow strip covers the curb face. Click **Rebuild reference**.
3. Scrub through the clip and watch the plot: dry and wet frames should stay above 0.45.
4. **Geometry ▸ Save…** → `site.json`. Enter the same values in the camera's web UI.

Test lighting changes (the reference gallery)

1. Pick a clip whose lighting changes, and a dry baseline from another time of day.
2. Device run with a short period, so a short clip gives enough samples:

```
floodsim run dusk.avi --reference noon.jpg --period 5 --out out/dusk
```

3. In `summary.txt`: *references learned* shows the gallery adapting on its own, and *no-confident-match notices* shows lighting it could not cover. In `samples.csv`, rows with `admitted=true` are where it learned, and `gallery_slot` shows which reference each sample then used.
4. If notices fire, add a baseline for that lighting (`--reference night.jpg`, or **Add reference** in the GUI) and run again: the notices should stop and correlation should rise.

The same works in the GUI: add a reference, set the period in **Run ▸ Device run…**, and watch the *slot / learned / notice / refs* columns in the samples table. Then run **Run ▸ Dense run…**: the curve now scores every frame against what the device run learned.

Find out why a frame was judged a flood

1. Click the dip in the plot to jump to it.
2. Look at the verdict panel:
   - a correlation below 0.45 means the curb strip changed;
   - a large `dy` means the camera moved.

3. Switch on **View ▸ Difference from reference** to see what changed, and **Show align** to check the alignment patch.
4. If a vehicle caused it, draw a box over it. If the box hides ≥ 15 % of the curb, the frame is vetoed.

9. When the firmware changes

- **Flood-detection C code** (`components/as_code/process_flood_detection/`): nothing to do. Firmware files are compiled in place, never copied, and are recompiled automatically the next time the simulator starts. If the DLL is locked by another open simulator window, close it and retry.
- **Device glue** (`flood_task.c`, `flood_frame_tap.c`, `flood_roi.h`, `flood_geometry_config.c`): these files depend on the RTOS and camera, so the simulator **copies** a few values from them instead of compiling them: the 60 s period, the grayscale formula, the default lines, the geometry checks, the upload scale, and the per-sample gallery bookkeeping order (skip when the gallery is empty; learn after each reading). All of them are listed, with their source file and line, in `native/device_constants.h`. If those firmware files change, compare that list by hand.

10. Troubleshooting

Symptom	Cause / fix
`firmware flood sources not found under …`	Set `FLOOD_FIRMWARE_DIR` to the firmware repo.
Build error mentioning `Permission denied`	Another simulator window has the DLL open. Close it.
`onnxruntime is not installed …` / `detection model not found …`	Install the `[onnx]` extra and put the model at `models/traffic.onnx` (or pass `--model`), or use `--no-detector`, or run `floodsim detect` elsewhere and copy the `.boxes.csv` over.
`… is not 16:9, so the curb geometry would not line up`	The input is not 16:9. Use `--allow-stretch` only if you know the lines still fit.
`image frames need --fps …`	The folder's file names have no `_t<ms>` part. Pass `--fps`.
`dense mode needs --reference or --reference-at`	Give a dry reference.
`… already exists and is not empty; choose another --out`	Runs never overwrite each other. Pick a new folder.
`geometry leaves no usable sampling window`	The lines overlap too little horizontally. Widen them.
`reference gallery is full of manual baselines`	8 manual references is the firmware's limit. Clear some (GUI: **Reference ▸ Clear all references**).
No references learned on a short clip	Too few samples at the 60 s period. Use `--period 5` (GUI: the period in **Run ▸ Device run…**).
Frames that look clearly dry are never learned	A manual reference must still score them ≥ 0.55 (the anchor). Strong shadows or night mode can score like a flood against every manual baseline: add a manual reference for that lighting.
`clip names must be record_YYYYMMDD_HHMMSS.<ext>`	Rename the listed files; the start time comes from the name.
`export does not support folders of clips`	Export a single clip instead.
Every device sample is `skipped_no_ref`	No reference was given.
Lots of `blocked` samples	The obstruction veto is firing. Look at the boxes in the GUI. With a single image or sparse stills, the "retry" re-reads the same frame, so it can't pass.
GUI status: "N frame(s) had no boxes"	No detector and no cache for those frames, so their veto is missing.
Video fails to open from a folder with Chinese characters in its path	An OpenCV limitation for videos. Move or rename the file. Images are fine.

11. Limits: where the simulator differs from the camera

- **Obstruction detector.** On the camera the traffic model runs on the IMX500 chip. Here the same model (`models/traffic.onnx`) runs on the PC in float, with the firmware's decoding, 0.5 threshold, NMS 0.20 and motorcycle filter, on the whole frame squeezed to 416×416. The chip runs a quantized build on its own crop of the sensor, so boxes and scores are close, not identical.
- **Retry timing.** The simulator retries 1 s after the first frame. The camera simply takes the next frame. The simulator may therefore clear a passing vehicle more often than the camera does.
- **First sample.** The simulator samples at t = 0, 60 s, 120 s…. The camera's first sample comes about 60 s after start.
- **Uploaded images.** They are the same frames, scale and ID, but encoded by OpenCV rather than the camera's JPEG hardware, so the files are not byte-identical.
- **Floating point.** The PC and the camera's RISC-V processor may round some single-precision maths slightly differently. This only affects borderline veto cases; the correlation maths is unaffected.
- **Not simulated:** web UI, MQTT, network uploads, OTA, persistent calibration storage.